Numerical Modelling using Metropolis Algorithm

Physics and Equations used: -

By paper method: -To find the exact average energy using pure physics, we are forced to look at every single place the particle could possibly ever be in the universe, calculate the odds of it being there, and average it all together.

1. Potential Energy: -

For a spring, the further you stretch it, the higher the energy goes. This creates a U-shaped "valley" of energy.

Physics equation used: - $$V(x) = \frac{1}{2}k_f x^2$$

Where k_f is the string constant of the spring and x is the distance, it stretched.

Python code replacement: - E = 0.5 * kf * (x * nm)**2 (uses simple algebra to give the particle an "energy score" depending on where it is standing at that exact millisecond)

2. The Boltzmann Distribution: -

The particle wants to sit at the bottom of the valley where the energy is zero. But the temperature kicks it up the sides. The **Boltzmann Factor** is the law of physics that dictates exactly how likely the particle is to survive jumping up to a high-energy spot.

Physics equation used: - $$Probability \propto e^{-\Delta E/k_B T}$$

Python code replacement: - if np.exp(-deltaE / kT) > np.random.rand():

3. Average Energy: -

Physics equation used: - $$\langle E \rangle = \frac{\int E \cdot e^{-E/k_B T} dx}{\int e^{-E/k_B T} dx}$$

Python code replacement: -

Inside the loop (doing this 20,000 times): Etot = Etot + E1

After the loop finishes: E_avg = Etot / N

4. Heat Capacity: -

Physics equation: - $$C_V = \frac{\langle E^2 \rangle - \langle E \rangle^2}{k_B T^2}$$

Python code replacement: - Cv = (E2_avg - E_avg**2) / (kb * T**2)

How the problem was solved: -

To model this in Python, we used the Metropolis algorithm. Here is how the code works step-by-step:

1. Assuming the particle at a random starting position and calculate its initial energy.
2. Moving the particle by a small, random amount.
3. Checking the Energy by using the Metropolis rule:
    - If the new position has lower energy, we accept the move immediately.
    - If the new position has higher energy, we don't automatically reject it. We calculate the probability of this happening using the Boltzmann equation. We then roll a virtual dice (a random number between 0 and 1). If the dice roll is less than the probability, we accept the move. Otherwise, we reject it and stay put.
4. Repeat: We repeat this process 20,000 times.
5. Average: Finally, we average the energy of all the accepted states to get our final answer. By favouring lower energies but allowing higher ones based on temperature, the algorithm perfectly samples the most important states.